

SANAD: Zero-PII Verifiable Credentials for Multilingual Government-Service Navigation in the UAE

Sanmeet Singh Kohli¹, Maria Mustafa¹, Sakshi Himpalnerkar², and Haitham Yash¹

¹ University of Wollongong in Dubai, Dubai, United Arab Emirates

² Maharashtra Institute of Technology, Chhatrapati Sambhajanagar, India
e-mail: {to be supplied by the authors}

Abstract. The UAE is a world-leading digital government, yet its largely expatriate residents face three barriers to using public services: navigating them across languages, understanding which documents and rules apply, and producing privacy-safe records of unresolved complaints such as unpaid wages. We present SANAD, a prototype assistant combining a schema-constrained large-language-model front end that maps free-text situations only to services in a curated catalogue of official UAE portals; an honest-degradation policy that labels every AI output by source and blocks sample fallbacks from credential issuance; and a zero-personally-identifiable-information (zero-PII) verifiable-credential mechanism that anchors each user-confirmed claim to a salted SHA-256 digest recorded by an issuer-controlled smart contract on the Polygon Amoy testnet. The chain attests only a record's existence and immutability, never its truth. A validation suite passes 12 of 12 automated tests, including live-PostgreSQL row-level-security isolation, and a grounded-matcher benchmark returns 16 of 16 correct matches with zero out-of-catalogue results.

Keywords: electronic government, verifiable credentials, blockchain, large language models, data minimisation, multilingual access, United Arab Emirates

1 Introduction

The United Arab Emirates consistently performs at the top of global digital-government assessments: the 2024 United Nations E-Government Survey places the UAE 11th worldwide on the E-Government Development Index with a score of 0.9533, and first in its e-Government Literacy sub-index [1] [2]. Availability of online services, however, is not the same as accessibility of outcomes. The UAE population is majority foreign-born (commonly cited at roughly 88%, an indicative figure), and residents routinely interact with services - labour complaints, visa and status questions, wage protection, unemployment insurance, tenancy disputes - whose entry points are spread across portals such as u.ae and the Ministry of Human Resources and Emiratization (MOHRE) [10].

Three gaps persist at the point of use. First, a language gap: official information exists in several languages, but guidance quality and user confidence diverge sharply

for residents whose strongest language is Hindi, Urdu or Bengali rather than Arabic or English. Second, an evidence and eligibility gap: residents frequently cannot tell which document proves what, which scheme they qualify for, or what a rejected application means. Third, a trust gap: when a dispute such as unpaid wages escalates, the worker's own records are informal, scattered and unverifiable to a third party, while any centralised attempt to build verifiable records collides with the UAE Federal Decree-Law No. 45 of 2021 on the Protection of Personal Data (PDPL) [7].

This paper describes SANAD, a working prototype that addresses all three gaps with one deliberately conservative architectural decision: personal data never leaves the user's own database partition, and what is published anywhere public - a blockchain - is only a salted cryptographic digest of a user-confirmed claim. SANAD is not a claim-detection system and does not assert that anything on-chain is factually true; it provides existence, immutability and timestamping proofs for records whose accuracy the user confirms.

Contributions. (1) A grounded navigation pipeline in which a large language model (LLM) may only select service identifiers from a curated catalogue of official portals, so that hallucinated services are structurally impossible in the matching path. (2) A zero-PII verifiable-credential mechanism: canonical serialisation, a 32-byte random salt and a domain-separated SHA-256 digest, anchored by an issuer-controlled Solidity registry, with verification that recomputes integrity off-chain and existence on-chain. (3) An honest-degradation policy for AI features - clearly labelled sample fallbacks at zero confidence that are blocked from credential issuance. (4) An automated validation suite, including row-level-security (RLS) tests against a live PostgreSQL instance, together with a published taxonomy of what is and is not yet proven (Sect. 6, Table 2).

Section 2 positions the work against prior art; Sect. 3 presents the architecture; Sect. 4 formalises the credential mechanism; Sect. 5 describes the grounded language layer; Sect. 6 reports implementation and validation evidence; Sect. 7 discusses limitations and ethics; Sect. 8 concludes.

2 Related Work

2.1 Blockchain-Backed Credentials

Since Nakamoto's proposal of an append-only, trust-minimised ledger [9], credential anchoring has become a recurring application: Blockcerts defined an open standard for blockchain-verified academic credentials issued by institutions [4], and later work surveyed blockchain approaches to credential management in education and beyond [5]. The W3C Verifiable Credentials Data Model gave the field a common representation for claims, proofs and subjects [3]. These systems typically put structured credential content - often including identifiers of the holder - into a signed artifact whose validity is checked against a ledger. SANAD takes the more restrictive route: no credential content is published at all. Only a digest computed over a canonicalised snapshot with a secret salt is recorded, so the ledger cannot be used to learn, correlate or enumerate anything about the holder.

2.2 Language Access and LLM-Grounded Public Services

Recent surveys of code-switched and multilingual natural-language processing emphasise that monolingual assumptions systematically degrade service quality for South Asian language communities, precisely the populations that dominate the UAE's expatriate workforce [6]. LLM assistants can lower the language barrier, but unconstrained generative systems are unsuitable for public-service guidance because they can invent plausible but non-existent schemes. SANAD constrains generation at the schema level: the model returns only catalogue identifiers and short rationales, and every identifier is joined back to a curated entry whose URL points to a government domain. Accessibility is treated as a first-class requirement following WCAG 2.2 [8], including right-to-left layouts for Arabic and Urdu.

2.3 Positioning

Existing gov-tech assistants optimise for conversational breadth; existing blockchain-credential systems optimise for portability of institutional attestations. SANAD targets the intersection that matters for resident-level disputes: a multilingual intake that is grounded by construction, and a trust artifact that is publishable without consent risk because it contains no personally identifiable information (PII). The PDPL principles of data minimisation and purpose limitation [7] guide the design: PII is confined to RLS-protected tables, and the public artifact is a hash.

Table 1. How SANAD differs from closely related approaches across the four properties that matter for resident-level, privacy-sensitive service navigation.

Approach	PII / content on-chain	Grounded to official catalogue	Honest-degradation policy	User-confirmed anchor
Blockcerts / institutional VC [3,4]	Signed credential content	No	No	Issuer attestation
Blockchain academic credentials [5]	Credential identifiers	No	No	Issuer attestation
Generic LLM gov-assistant	None, but no anchor	No (hallucination risk)	No (silent fallbacks)	None
SANAD (ours)	None (salted digest only)	Yes (28-entry catalogue)	Yes (labelled, 503)	Yes (user-confirmed)

3 System Architecture

SANAD is a full-stack web application: a Next.js 16 (App Router) front end with TypeScript in strict mode, server route handlers as the only trusted API boundary, Supabase (PostgreSQL with row-level security and the @supabase/ssr session helper) for persistence, Gemini accessed through the Vercel AI SDK's generateObject call with a Zod schema, and ethers v6 for transaction signing against the

SANADCredential contract on Polygon Amoy (chain ID 80002). Figure 1 shows the pipeline end to end.

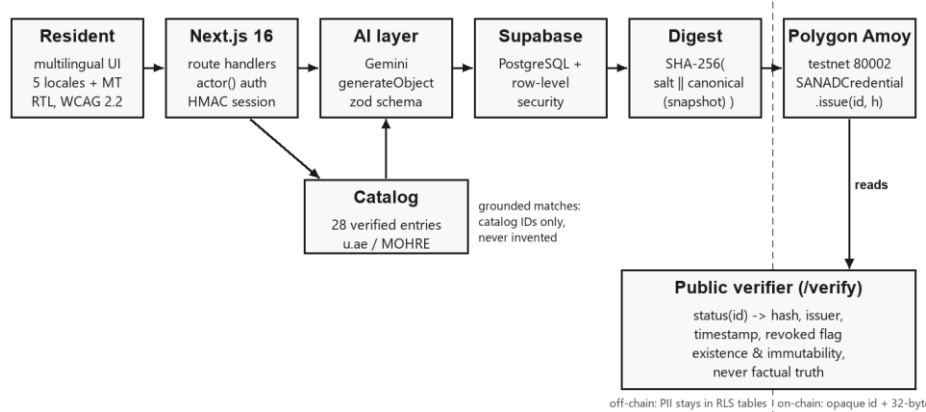


Fig. 1. End-to-end architecture. Personal data stays left of the dashed boundary in RLS-protected storage; only an opaque identifier and a 32-byte digest cross onto the testnet. The public verifier reads existence and revocation status, never content.

A session begins either with Supabase authentication or with a signed demo cookie (HMAC-SHA-256 over an identifier and expiry, httpOnly, SameSite=Lax). Every mutating route handler resolves the caller through a single actor() helper before touching data; unauthenticated requests receive 401 and the interface responds with an explicit sign-in prompt rather than silently showing another user's data. The user describes a situation in their own language; the AI layer extracts structured facts and matches the situation against the official-services catalogue; document analysis (e.g. a wage slip or contract photo) can attach evidence summaries; the user then confirms or edits the resulting claim statement. Only a confirmed claim is snapshotted, hashed and - in real mode - issued on-chain. The catalogue itself is data, not model memory: 28 curated entries, each linking to u.ae or mohre.gov.ae, with supported-situation tags used for deterministic pre-filtering before the model is consulted.

The interface ships with five hand-curated locales - English, Arabic (RTL), Hindi, Urdu (RTL) and Bengali - plus machine-translated coverage of additional resident languages via an in-page translator, so the intake path is usable without English proficiency. Figure 2 shows four production screens captured from the running prototype.

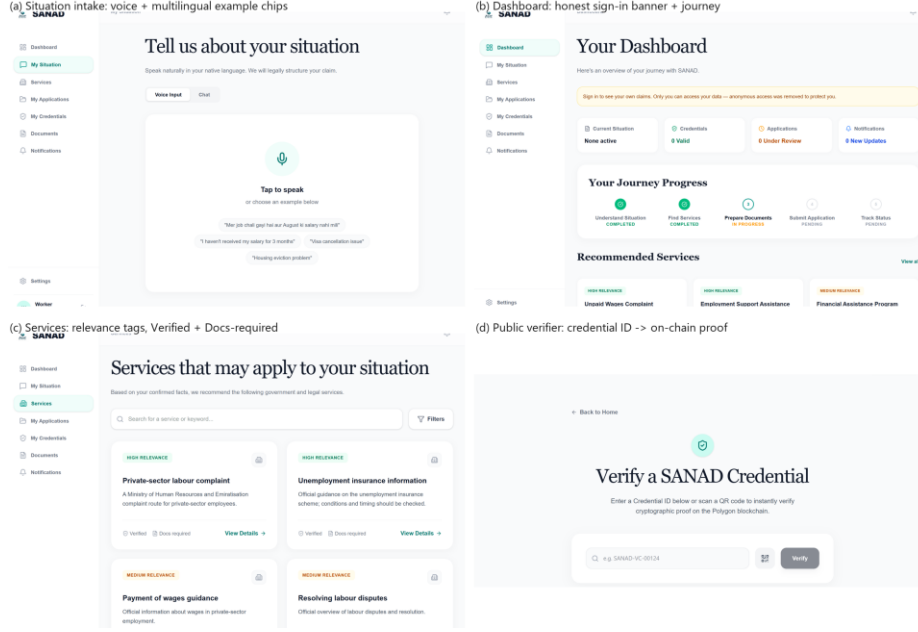


Fig. 2. Prototype screens (captured live, not mock-ups). (a) Situation intake with voice input and multilingual example prompts; (b) dashboard with the explicit sign-in banner that replaced anonymous claim access and the five-stage journey tracker; (c) grounded service list with relevance, Verified and Docs-required tags linking to official portals; (d) the public verifier that resolves a credential ID to an on-chain existence proof.

4 The Zero-PII Credential Mechanism

4.1 Canonicalisation and Salted Digest

When a user confirms a claim, the server builds an immutable snapshot (statement, extracted facts, provenance flag `USER_CONFIRMED`), serialises it with a canonical encoder that sorts object keys and recursively normalises arrays and scalars, and computes

$$h = \text{SHA-256}(\text{"SANAD:v1:"} \parallel \text{salt} \parallel \text{"."} \parallel \text{canonical(snapshot)})$$

where salt is 32 fresh random bytes per credential. The domain-separator prefix ("SANAD:v1:") prevents cross-protocol collisions; the salt makes the digest non-malleable and non-dictionary-matchable, since an observer who guesses the snapshot contents still cannot reproduce h without the salt. Both snapshot and salt are stored only in RLS-protected off-chain tables; the pair (credential identifier, h) is what crosses the trust boundary. A UUID is mapped to a bytes32 identifier by hex-padding or, failing that, SHA-256, so no user-facing identifier appears on-chain either.

4.2 Issuer-Controlled Registry

The on-chain component is deliberately minimal: a single contract whose issuer key is immutable, with issue, revoke and a free status view (Listing 1). Duplicate issuance for an identifier is rejected; revocation is one-way; the stored record hash is never overwritten, giving tamper-evidence against any later change of the off-chain snapshot.

```
contract SANADCredential {
    address public immutable issuer;

    struct Credential { bytes32 recordHash; address
issuedBy;

                                uint64 issuedAt; bool revoked;
}

    mapping(bytes32 => Credential) private records;
    event Issued(bytes32 indexed id, bytes32 indexed
recordHash,

                                address indexed issuer);

    modifier onlyIssuer() {
        if (msg.sender != issuer) revert NotIssuer();
    _; }

    function issue(bytes32 id, bytes32 recordHash)
external onlyIssuer {
        if (records[id].issuedAt != 0) revert
AlreadyIssued();

        records[id] = Credential(recordHash,
msg.sender,

uint64(block.timestamp), false);

        emit Issued(id, recordHash, msg.sender);
    }

    function status(bytes32 id) external view
        returns (bytes32, address, uint64, bool) {
        Credential storage e = records[id];

        return (e.recordHash, e.issuedBy, e.issuedAt,
e.revoked);
    }
}
```

Listing 1. Core of the deployed SANADCredential registry (Solidity ^0.8.24, MIT).

4.3 Verification Semantics and What They Do Not Prove

Verification composes two independent checks. Integrity: recompute h from the stored snapshot and salt and compare with the stored record hash. Existence: read `status(id)` on the chain and confirm the returned hash matches, the timestamp is present and the revoked flag is false. A credential is VALID only if both hold. Crucially, the chain attests that a given digest existed at a given time and has not been altered - it cannot attest that the underlying statement about the world is true. SANAD encodes this limit in the product itself: verification pages and issuance dialogs state the distinction, and the paper throughout uses “verifiable” only in the cryptographic sense.

4.4 Threat Model, Issuer-Key Risk and Retention

We make three adversary classes explicit. A chain observer sees only an opaque identifier and a 32-byte digest; because the digest is salted with a per-credential 32-byte secret held off-chain, guessing the snapshot contents is insufficient to reproduce it, so the observer cannot learn, correlate or enumerate holders or claims. A database adversary who defeats row-level security can read snapshots and salts, but still cannot forge a valid on-chain anchor without the issuer key, and any silent edit of a stored snapshot breaks the integrity recomputation in Sect. 4.3. A holder of the issuer key can issue or revoke records; this is the principal trust assumption. It is not mitigated away - the issuer address is immutable and every issue/revoke emits a public event, so abuse is attributable and visible rather than impossible, and production use should replace the single key with a multisig or HSM and a documented revocation policy.

Retention and the zero-PII scope are stated narrowly. Snapshots and salts persist server-side under RLS until a user deletes them; deletion removes the off-chain data, which renders the on-chain anchor permanently unverifiable - a deliberate right-to-erasure property that also means an anchor cannot outlive the consent that created it. Finally, “zero-PII” is a statement about the public artifact, not a legal conclusion: the design is consistent with the PDPL principles of data minimisation and purpose limitation [7], but we do not claim that it, by itself, establishes regulatory compliance.

5 Grounded Language Understanding and Honest Degradation

Two AI endpoints carry user-visible risk: situation understanding (free text or voice transcript to structured facts and service matches) and document analysis (photo or PDF to extracted fields and risk flags). Both are schema-constrained: `generateObject` is given a Zod schema and a prompt that embeds the catalogue and forbids inventing services; returned identifiers are joined back to catalogue entries and any identifier not present in the catalogue is dropped before rendering. The user therefore only ever sees official, linkable services.

The honest-degradation policy governs failure. If no model key is configured, the endpoint returns 503 rather than pretending to work. If the provider call fails, the response is constructed from a clearly labelled SAMPLE object with confidence 0 and a source field set to `sample_fallback`; the UI renders a visible warning banner, and the issuance path refuses to anchor a credential whose analysis source is `sample_fallback`. This converts a common demo shortcut - fabricated output indistinguishable from real output - into a state that is machine-blocked and human-visible, which we consider a minimum ethical bar for civic AI prototypes.

6 Implementation and Validation

The repository implements 24 application routes (13 statically prerendered, 11 server-rendered), the credential contract, the catalogue as typed data, and an automated test suite. At submission time the following evidence holds, and Table 2 records what each result does and does not support.

Static and build checks: TypeScript strict type-check passes; ESLint reports zero errors (ten warnings); next build completes for all 24 routes. Functional tests: the Vitest suite passes 12/12, including RLS isolation tests executed against a live PostgreSQL instance that assert one user cannot read or write another user's claims and credentials. Contract checks: local harnesses verify issuer-only issue and revoke, immutability of the stored hash, duplicate-issuance rejection and revocation semantics; a preflight against the Amoy testnet confirms the deployed ABI matches the client (`issue(bytes32,bytes32)`, `revoke(bytes32)`, `status(bytes32)`). Security review remediation: previously committed credentials were removed from the working tree and HEAD, unauthenticated claim reads were closed by enforcing `actor()` plus per-user filtering on every claims path, and fabricated AI fallbacks were replaced by the labelled-sample policy of Sect. 5. Secrets that had entered git history remain a rotation obligation, tracked as pending on the authors' side.

6.1 Grounded-Matcher Benchmark

To move beyond the qualitative grounding argument of Sect. 5, we exercised the live production endpoint (`POST /api/services/match` on the built server, same prompt, Zod schema and 28-entry catalogue as the shipped app) over 16 free-text situations spanning five input languages: English, Hinglish, Roman Urdu, Arabic script, Devanagari Hindi and Bengali. Each case carries an acceptable service-id set derived from the catalogue's own situation tags, so ground truth is reproducible from repository data rather than hand-asserted. We report `hit@1` (top match acceptable), `hit@3` (any of the returned matches acceptable) and the count of out-of-catalogue identifiers. Result: 16/16 `hit@1`, 16/16 `hit@3`, 0 out-of-catalogue ids, 2.63 matches per case and a median latency of 1378 ms (range 1056-2120 ms). Every returned identifier resolved to a real official service, confirming the structural grounding claim end to end. We report this as a small, single-annotator, catalogue-derived benchmark: it measures retrieval correctness against curated ground truth, not real-world user

outcomes, and two transient provider 500s observed during development (both recovered on retry) are reported as provider-reliability noise rather than hidden.

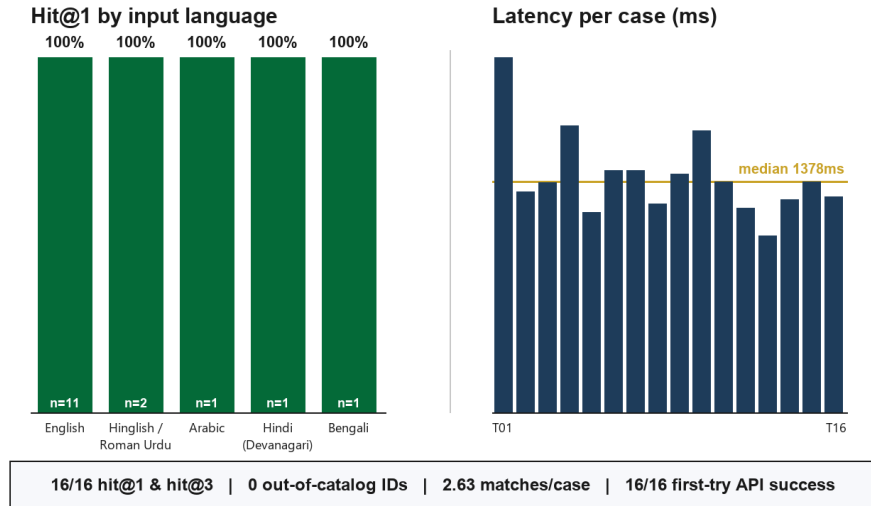


Fig. 3. Grounded-matcher benchmark. Left: hit@1 is 100% in every input-language group (case counts annotated). Right: per-case end-to-end latency with median line. Footer reports the aggregate metrics from the live endpoint run.

6.2 Claim Ledger

Table 2. Validation status at submission. Rows marked “not claimed” are reported to bound the paper's claims, not to criticise them.

Mechanism	Evidence at submission	Status
Data isolation (RLS)	12/12 Vitest incl. live-PostgreSQL isolation tests	Proven (prototype)
Registry semantics	Issuer-only, duplicate-reject, one-way revoke, immutable hash	Proven (tests + Amoy preflight)
Catalogue authenticity	28 entries, every URL on u.ae or mohre.gov.ae	Proven URLs; freshness unverified
AI grounding (retrieval)	Schema-constrained ids; 16/16 benchmark hit@1, 0 out-of-catalogue	Proven on curated benchmark; user outcomes untested
Honest degradation	503 without key; SAMPLE at confidence 0; issuance blocked	Proven (code + UI)
Factual truth of claims	Chain attests existence/immutability only	Not claimed
Production operation	Testnet only; key rotation pending	Not claimed
Resident usability	No user study conducted	Not claimed

7 Limitations and Ethical Considerations

We state the boundaries plainly. The chain layer runs on a public testnet with a single issuer key; production use requires mainnet deployment, key custody (HSM or multisig), and a revocation policy agreed with the institutions that would honour these anchors. The AI layer's benchmark (Sect. 6.1) is small, single-annotator and scored against catalogue-derived ground truth: it evidences retrieval correctness, not real-world user outcomes, and document extraction quality has still not been measured on real, consented documents. The credential mechanism anchors user-confirmed statements; it adds no independent verification, and an adversary with the issuer key can anchor false statements - the immutable issuer address and public events make such abuse visible and attributable, not impossible. The demo mode's signed cookie is a hackathon convenience, not an identity system. Finally, the headline demographic figure used in outreach (about 88% foreign-born) is indicative and cited as such; all product claims in this paper are bounded by Table 2.

Ethically, the design follows data minimisation by construction: PII remains in user-partitioned tables under RLS, nothing personal is sent to the blockchain or to the model provider beyond the transient document image used for extraction, and the honest-degradation policy exists specifically so that a demonstration can never be mistaken for an operational service. Accessibility (WCAG 2.2, RTL support) and the PDPL-aligned storage model are treated as functional requirements rather than polish.

8 Conclusion

SANAD shows that the three barriers residents face in using even a world-class e-government - language, evidence clarity and trust - can be attacked together by a small system whose core commitments are negative: the model never invents services, the chain never sees personal data, and the product never claims more than its proofs support. The prototype passes its automated validation suite, anchors digests on a public testnet, and ships an explicit proven/not-proven ledger as part of its public surface. Next steps are a 90-day innovation pilot with a government partner: consented usability testing with service-seekers, accuracy benchmarking of the grounded matcher, mainnet custody, and integration of an institutional verifier that can check anchors without access to the off-chain snapshot. We invite evaluation of the system by exactly the standard it sets for itself: every claim verifiable, and none overstated.

References

- [1] United Nations Department of Economic and Social Affairs: E-Government Survey 2024. United Nations, New York (2024)
- [2] Government of the United Arab Emirates: United Arab Emirates - e-Government Development. [u.ae, https://u.ae/en/about-the-uae/digital-uae](https://u.ae/en/about-the-uae/digital-uae) (accessed 2026)

- [3] World Wide Web Consortium: Verifiable Credentials Data Model 1.0 - W3C Recommendation. <https://www.w3.org/TR/vc-data-model/> (2019)
- [4] MIT Digital Currency Initiative: Blockcerts - An Open Standard for Blockchain-Verified Academic Credentials. <https://www.blockcerts.org/> (2017)
- [5] Bapat, P., et al.: Blockchain for Academic Credentials. arXiv:2006.12665 (2020)
- [6] Sheth, R., et al.: Beyond Monolingual Assumptions: A Survey of Code-Switched NLP in the Era of Large Language Models across Modalities. arXiv:2510.07037 (2025)
- [7] United Arab Emirates: Federal Decree-Law No. 45 of 2021 on the Protection of Personal Data and Privacy (2021)
- [8] World Wide Web Consortium: Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/> (2023)
- [9] Nakamoto, S.: Bitcoin: A Peer-to-Peer Electronic Cash System (2008)
- [10] UAE Ministry of Human Resources and Emiratisation: Register labour complaints for private-sector employees. <https://mohre.gov.ae/en/services/register-labor-complaints-private-sector-employees-2022> (accessed 2026)